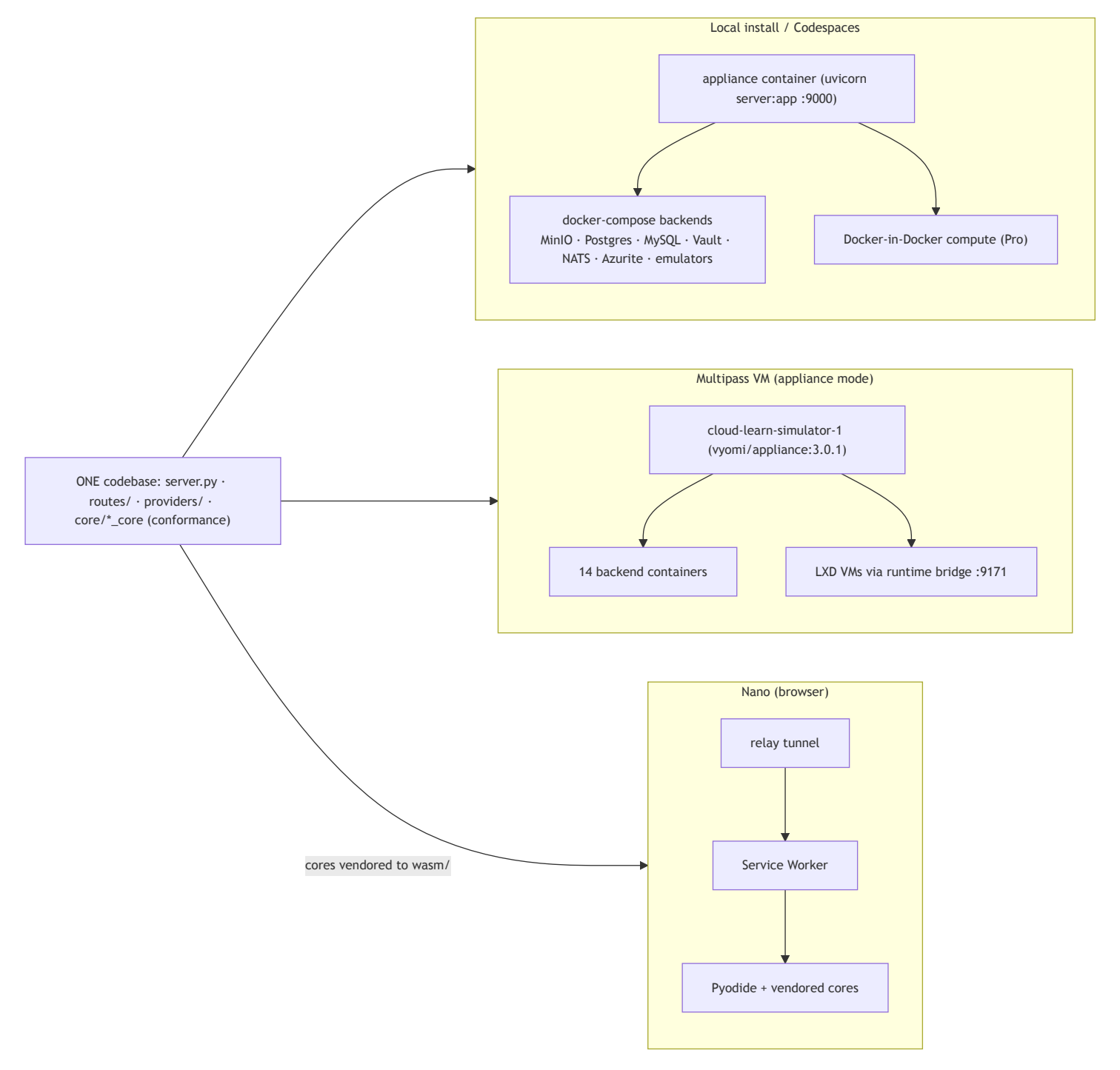
Vyomi Appliance — Detailed Component Architecture

A full-blown component diagram of the appliance (v3.0.x). It shows the request paths (console REST **and** native SDK/CLI wire), the provider + conformance-core layers, the store **seams** with their in-memory vs real-backed implementations, the real backends, the compute/runtime path, support systems, and the Nano/WASM substrate that reuses the same cores. `console-next` (new UI) is marked PARKED (feature branch).

① Clients
Native SDK / CLI boto3 · gcloud · az · terraform (SigV4 / ARM / REST wire)
Browser — current consoles static/aws|gcp|azure-console.html
console-next SPA (PARKED · feat/console-redesign)
vyomi launcher CLI
SigV4 / ARM / REST
/api/*
/api/console/* + native
native wire
② Entry — server.py · FastAPI app :9000
app = FastAPI()
registration loop routes/*.register(app) · providers register
VYOMI_UNIFIED_INGRESS?
off
on
ASGI middleware (core/middleware.py) TenantContext · HeaderAlias (X-CloudLearn→X-Vyomi) CORS · GlassBoxCapture tap
S3 native-wire catch-all /{bucket}/{key:path} (registered last)
device-flow / activation /api/auth/*-activation
⑪ Nano substrate (wasm/) — same cores, in-browser
wasm/relay/* · external SDK ↔ tab tunnel
fixtures (build_fixtures.py)
Nano mode
browser substrate
sw.js · fetch→Pyodide ASGI shim
nano-boot.js · Pyodide boot
wasm/providers/*_adapter.py
wasm/core/* (vendored cores via build_cores.py) + aws_wire_router
⑤ Native wire ingress (opt-in · VYOMI_UNIFIED_INGRESS)
core/wire_ingress.py build_backed_router() · mount()
core/aws_wire_router.py SigV4 cred-scope → X-Amz-Target → core
③ Console REST route modules (routes/ + /api/* in server.py)
aws_s3 · /api/s3/*
aws_ec2 · /api/ec2/*
aws_rds · /api/rds/*
aws_dynamodb · /api/dynamodb/*
aws_sqs/sns · /api/sqs,/api/sns
aws_lambda · /api/lambda/*
aws_vpc · aws_apigw
gcp_console · /api/gcp/*
azure_console · /api/azure/*
cloudsim · spaces · tenants config · licensing · runtime · terraform lazy_backends · shared
console_next · /console-next + /api/console/* (PARKED)
④ Provider registries (providers/)
aws.py + aws_ec2_routes
registry.py CloudProvider seam + register()
gcp.py + gcp_routes + gcp_compute_routes
azure.py + azure_services + azure_arm_core
catalogs + capabilities aws/gcp/azure-catalog · fixtures
⑥ Conformance cores (core/*_core.py) — substrate-free, same logic on every substrate
Azure cores
azure_blob_core · azure_queue_core
azure_sql_core · azure_cosmos_core
azure_servicebus_core
azure_keyvault_secrets/keys_core
azure_iam_core
azure_arm_core (ARM dispatcher)
GCP cores
gcp_storage_core
gcp_cloudsql_core
gcp_firestore_core
gcp_pubsub_core
gcp_kms_core · gcp_secretmanager_core
gcp_iam_core
AWS cores
s3_core
dynamodb_core
rds_core · rds_data_core
sqs_core · sns_core
kms_core
secrets_core
iam_core (AuthZ eval)
vpc_core · apigateway_core
lambda_core · eventbridge_core
⑦ Store seams (core/) — in-memory default ⟷ real-backed impl
ObjectStore InMemory ⟷ MinioBackedObjectStore / minio_mirror
SqlStore sqlite ⟷ PostgresBackedSqlStore
NoSqlStore InMemory ⟷ Persistent* (SQLite)
MessagingStore InMemory ⟷ NatsBackedMessagingStore
KvStore / KeyStore InMemory ⟷ VaultBacked*
IamStore InMemory (principals + policies)
app_context.py in-memory state proxies (space-scoped)
⑧ Compute / runtime
core/compute · ComputeBackend ABC DockerComputeBackend / LXD / Multipass
runtime bridge CLOUDLEARN_RUNTIME_BRIDGE_URL :9171
vm_connect.py SSH key inject · lxc proxy · connect-info
⑩ Support systems (core/)
install_registration.py phone-home telemetry (cc geo)
glassbox.py · call-event ring buffer + SSE
persist_state / load_state · integrity · security_audit
license + jwt_signer + license_remote tier JWT · activation
tier_policy.py category gating
shared_tenancy.py cohort / namespace isolation
cloudsim · cost/usage meter (→ :9010 service)
backend_provisioner.py lazy backend recipes
backed
backed
backed
backed
phone-home
JWKS / activate
⑨ Real backends (containers / host)
Postgres (RDS/CloudSQL/AzureSQL)
MySQL
Firestore emulator
Pub/Sub emulator
NATS JetStream
MinIO (S3/GCS bytes)
ElasticMQ (SQS)
Vault (KMS/Secrets Transit+KV)
Vyomi Portal (license JWKS · activation · install telemetry)
cloudsim service :9010 (cost/usage sim)
LXD / Multipass / Docker (VMs)
Azurite (Azure Blob)

Topology — the three substrates (one codebase)



ONE codebase: server.py · routes/ · providers/ · core/*_core (conformance)
Local install / Codespaces
appliance container (uvicorn server:app :9000)
docker-compose backends MinIO · Postgres · MySQL · Vault · NATS · Azurite · emulators
Docker-in-Docker compute (Pro)
Multipass VM (appliance mode)
cloud-learn-simulator-1 (vyomi/appliance:3.0.1)
14 backend containers
LXD VMs via runtime bridge :9171
Nano (browser)
cores vendored to wasm/
relay tunnel
Service Worker
Pyodide + vendored cores

Notes

- **Two request paths converge on the same cores:** the **console REST** routes (what the UI calls) and the **native SigV4/ARM/REST wire** (what real SDKs call). Today the console REST writes to in-memory `app_context` state; the **backed stores** (MinIO/Postgres/Vault/NATS) are hit by the wire path when `VYOMI_UNIFIED_INGRESS` is on. Unifying those is the parked backed-store work.
- **Compute is already real:** EC2/GCE/AzureVM create → `ComputeBackend` → runtime bridge → a real **LXD** container.
- **Nano = same cores, different transport:** the Service Worker is the ASGI shim; Pyodide runs the *vendored* conformance cores; the relay tunnels external SDKs into the tab. No fork of the service logic.
- **One seam per capability, shared across clouds:** the AWS, GCP and Azure cores for the *same capability* resolve to the **same store seam** — e.g. `s3_core`, `gcp_storage_core`, `azure_blob_core` all land on `ObjectStore` →MinIO; `rds_core` / `gcp_cloudsql_core` / `azure_sql_core` on `SqlStore` →Postgres; `sqs/sns_core`, `gcp_pubsub_core`, `azure_servicebus_core` on `MessagingStore` →NATS. The cloud-specific wire fidelity lives in the core; the storage is unified underneath.
- **Glass-box capture ships in main:** `GlassBoxCaptureMiddleware` (core/middleware.py) taps every request into the `glassbox.py` ring buffer + SSE stream. The console-next *Inspector widget* that visualizes it is what's parked — the capture backbone is live.